
Appendix A: Assessment cover

**Assessment cover**

STUDENTS, PLEASE COPY THIS PAGE AND USE AS THE COVER PAGE FOR YOUR SUBMISSION

Module No:	COMP4035	Module title:	Computer Science Applications
------------	-----------------	---------------	--------------------------------------

Assessment number:	Coursework	Assessment title:	Artificial Intelligence – Decision Tree Learning
--------------------	-------------------	-------------------	---

Banner assignment identifier	CWS2WEEK6	Due date and time:	03/03/2026, 1 pm
------------------------------	------------------	--------------------	-------------------------

Estimated total time to be spent on assignment:	<i>30 hours per student direct work on coursework, plus 27 hours per student on independent study</i>
---	---

LEARNING OUTCOMES

On successful completion of this module, students will be able to achieve the module following learning outcomes (LOs):

LO#4 Understand principles of search, decision trees, and apply simple classification algorithms

Statement of Compliance (please tick to sign)

I declare that the work submitted is my own and that the work I submit is fully in accordance with the University regulations regarding assessments (www.brookes.ac.uk/uniregulations/current)

Use of AI Tools: You are required to use this [form](#) to declare which AI tools you have used and how you have used them. Please complete the form and attach it to your submission as an Appendix, if you have used such tools.

Google collab: <https://colab.research.google.com/drive/1HX-2wLoOVXRdmpHuucI77L6eqes8kky8?usp=sharing>

Decision trees are supervised machine learning models most often applied to classification problems. They operate by dividing data into branches according to feature values, creating a tree-shaped sequence of decisions. A key strength of decision trees is their interpretability, especially when compared with more complex approaches like neural networks.

In current real-world systems, decision trees appear in fields such as healthcare and finance. In medical triage, for instance, they can sort patients based on symptoms and risk levels. In banking, they support credit scoring by estimating whether a borrower is likely to repay a loan. In both domains, the clarity of the model's reasoning is a major advantage.

The Gini index (or Gini impurity) is a metric used to assess how "pure" a node is within a decision tree. A Gini value of 0 indicates that all samples in the node belong to one class, while higher values show that classes are mixed. During training, the algorithm selects the split that most reduces the weighted Gini impurity. As a result, the Gini index directly shapes how the tree determines its branching points.

Dataset Acquisition and Preparation

Dataset Overview:

The Car Evaluation dataset was obtained from the UCI Machine Learning Repository. It contains 1,728 instances and 7 attributes. The features describe different characteristics of cars, including buying price, maintenance cost, number of doors, number of persons, luggage boot size, and safety rating. The target variable represents the overall evaluation of the car, with classes such as unacceptable, acceptable, good, and very good.

Shape: (1728, 7)

Missing values:

buying0

maint0

doors0

persons0

lug_boot0

safety0

class0

dtype: int64

Class distribution:

class

unacc1210

acc384

good69

vgood65

Name: count, dtype: int64

	buying	maint	doors	persons	lug_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

After inspecting the dataset, it was confirmed that there were no missing values. All features were categorical, meaning they contained text values instead of numbers. The class distribution shows that the “unacceptable” class appears much more frequently than the others. This indicates that the dataset is imbalanced. Because of this imbalance, the model may become biased towards predicting the majority class more often. Minority classes such as “good” and “very good” may therefore be harder to classify correctly.

Data Preprocessing:

Since all features were categorical, label encoding was applied to convert the text values into numerical values. This step is necessary because decision tree implementations in Python require numerical input rather than strings. Label encoding assigns each category a unique integer value.

The dataset was then split into 80% training data and 20% test data using stratified sampling. Stratified splitting ensures that the class proportions remain similar in both the training and test sets. This is important because it prevents one dataset from being dominated by a single class. The 80/20 split is widely used in machine learning because it balances having enough data for training while still allowing a fair evaluation of model performance on unseen data.

Experiment Design and Implementation

Decision Tree using Scikit-Learn:

- Comparing Tree Depth

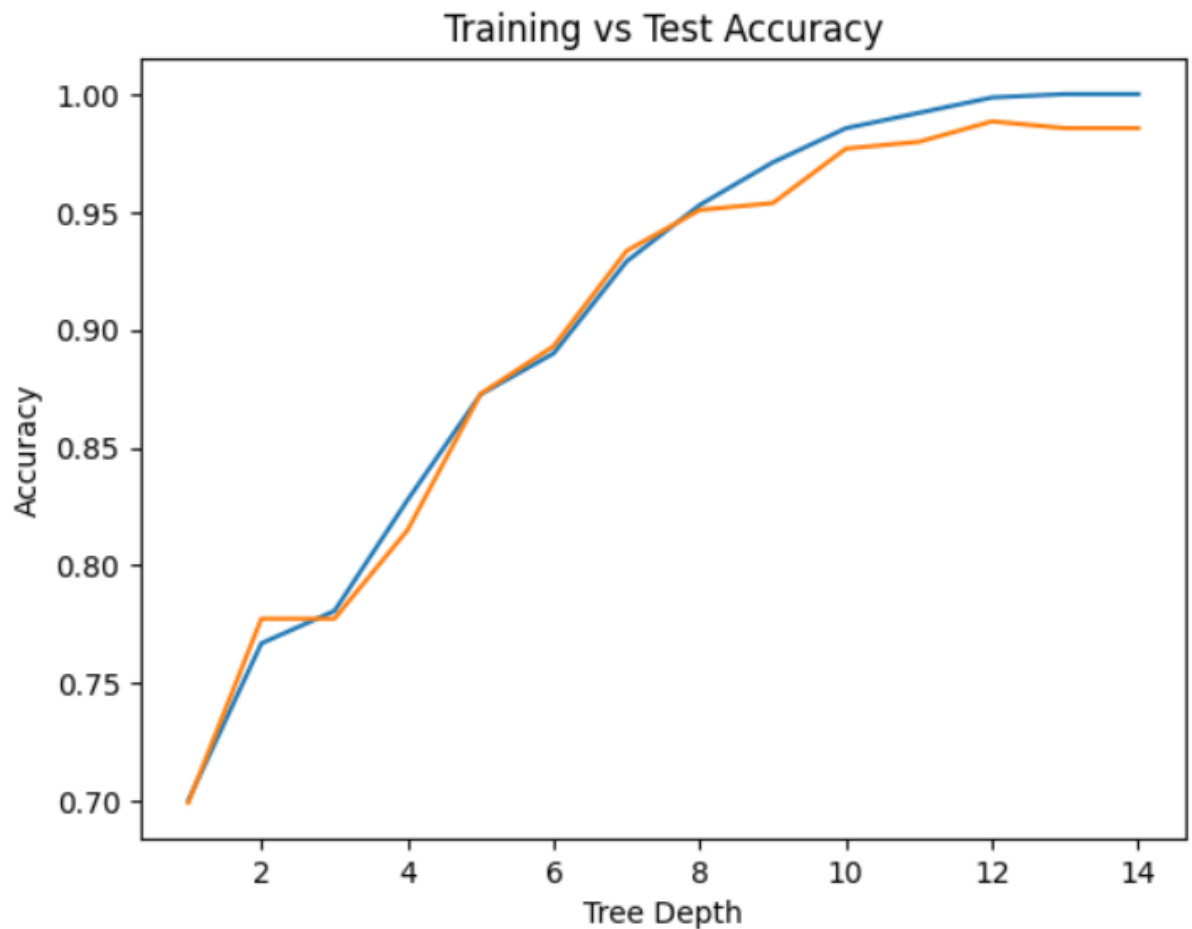
Two decision tree models were trained using the scikit-learn library. The first model was restricted to a maximum depth of 2, making it a shallow and simple tree. The second model had no depth restriction, allowing it to grow fully until nodes became pure or no further splits were possible.

The results were:

- Accuracy (Depth = 2): 0.777
- Accuracy (Full Tree): 0.986

The shallow tree ended up with an accuracy of about 77.7%, which is pretty decent for a model that simple. The fully grown tree, meanwhile, hit 98.6% accuracy, which is a huge jump. But numbers that high can be a warning sign, because it often means the model has clung too closely to the training data and might not handle new, unseen examples as well, a classic case of overfitting

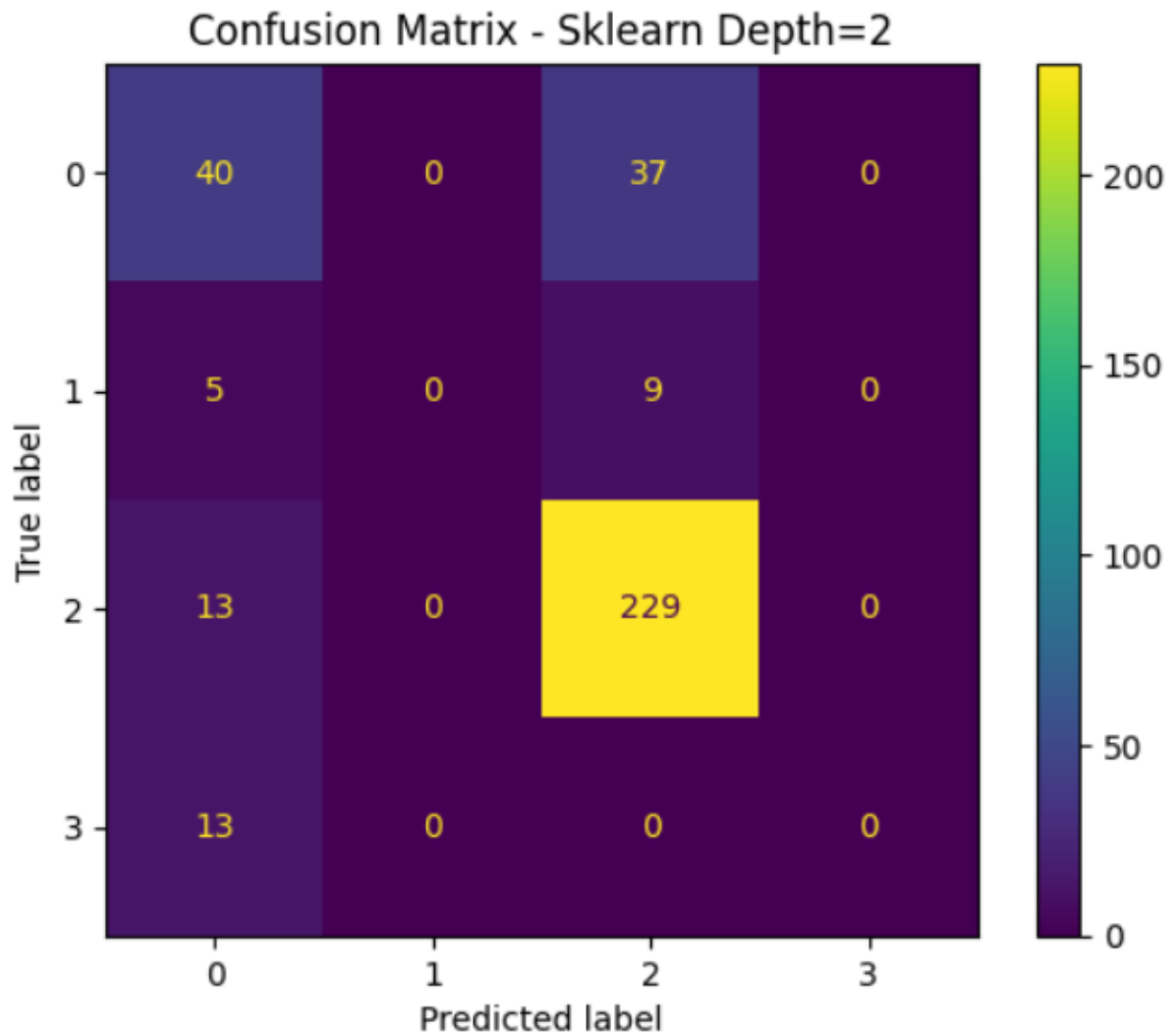
- Accuracy vs Tree Depth



The accuracy vs depth graph shows that training accuracy increases steadily as depth increases, eventually reaching almost 100%. This is expected because deeper trees can create more specific splits. However, the test accuracy increases more slowly and levels off after a certain depth. This difference between training and test accuracy suggests that deeper trees begin to overfit. In other words, they memorise patterns in the training data that do not necessarily generalise well.

This experiment clearly demonstrates the trade-off between simplicity and performance.

- **Confusion Matrix**



The confusion matrix lays out how many items the model got right or wrong for each class. You can see straight away that it does a stronger job on the majority class than on the minority classes. That's not surprising given the imbalance in the data. A number of minority-class examples end up being predicted as the majority class, which hurts the accuracy for those groups.

Gini Index Calculation:

The Gini index was implemented manually using the formula:

$$\text{Gini} = 1 - \sum(p^2)$$

where p represents the probability of each class within a node.

The Gini impurity of the full training dataset before splitting was:

0.457

```
Gini of training set: 0.4570454112310227
```

This value suggests that the dataset starts with a moderate amount of class mixing before any splits are made, meaning the classes are not perfectly separated at the root. Because of this, the decision tree's job is to gradually reduce that impurity by choosing features that create cleaner, more distinct groups at each step. As the tree grows, each split aims to push the data toward nodes that contain more consistent class labels, improving the model's ability to make accurate and meaningful predictions.

- Determining the Best First Split

To find the best first split, I calculated the weighted Gini impurity for every possible feature and split value. The feature that resulted in the lowest weighted Gini impurity was selected as the root node.

The best first split was:

Feature: safety (feature index 5)

Split value: 1

Gini after split: 0.385

```
Best feature index: 5  
Best split value: 1  
Gini after split: 0.38499511557696947
```

The impurity decreased from 0.457 to 0.385 after splitting on the safety feature. This shows that safety plays a very important role in determining car quality. Logically, this makes sense because safety is a critical factor in evaluating whether a car is acceptable or not.

- Custom Model Performance

The custom depth-2 tree achieved an accuracy of 0.699

The custom model achieved lower accuracy compared to both sklearn models. This is expected because the custom implementation only performs one main split and does not grow deeper. As a result, it cannot separate classes as effectively as the unrestricted sklearn tree.

- Challenges

One of the main challenges was correctly calculating the weighted Gini impurity for each possible split. It was important to ensure that the subsets created by each split were not empty. Another difficulty was implementing the majority class selection correctly for each branch.

Although the implementation was relatively simple, it required careful attention to indexing and logic. However, building the decision tree manually greatly improved my understanding of how the algorithm selects features and reduces impurity step by step.

Conclusions and Discussion

The unrestricted decision tree tends to overfit because it keeps splitting the data until the nodes become almost completely pure. This means it continues creating more and more detailed rules that fit the training data very closely. While this leads to extremely high training accuracy and a very high test accuracy of 98.6%, it also makes the model much more complex. A very deep tree can start learning very specific patterns that may not generalise well to new unseen data. The accuracy vs depth graph clearly shows this behaviour: training accuracy steadily increases as the tree gets deeper, but test accuracy improves more slowly and eventually levels off. This difference between the two curves suggests that after a certain depth, the model is learning noise rather than useful patterns.

The first split selected by the custom implementation was the safety feature. This makes sense logically, because safety is likely to be one of the most important factors when evaluating a car. A car with low safety is unlikely to be rated highly overall. Since both my custom implementation and sklearn use Gini impurity to measure split quality, it is not surprising that safety was selected early in the tree-building process. The reduction in Gini impurity from 0.457 to 0.385 shows that splitting on safety significantly improved class separation.

Building the decision tree from scratch helped me understand much more clearly how decision trees actually work behind the scenes. Instead of just calling a library function, I had to manually calculate impurity values and compare possible splits. This made the algorithm feel less like a “black box”. However, my implementation was much simpler than sklearn’s version and did not include optimisations such as pruning or automatic stopping rules. As a result, it was less accurate and less flexible.

In healthcare applications, shallow trees are often preferred because they are easier to interpret and explain. Doctors and healthcare professionals need to understand why a model makes a certain prediction. Even if a deeper tree achieves slightly higher accuracy, a simpler tree may be more practical because its decision rules are

clear and transparent. In high-risk environments, interpretability and trust can be just as important as raw predictive performance.